

On-Device Network Traffic Anomaly Detection Using TinyML

MLP301 – Machine Learning Principles

Assessment 2:

Name: Tia Darvell

Student ID: A00029275

Torrens University Australia

Lecturer: Subhash Sagar

Submission Date:

Executive Summary

On-device network traffic anomaly detection represents a compelling application of Tiny Machine Learning (TinyML) at the intersection of cybersecurity and edge computing. This proposal outlines the development of a machine learning (ML) model capable of classifying network traffic as normal or anomalous, deployed on a Raspberry Pi 2 single-board computer at zero additional hardware cost. Using Edge Impulse as the TinyML-as-a-Service (TinyMLaaS) platform, the model will be trained on a carefully selected subset of the publicly available UNSW-NB15 dataset, leveraging neural network classification, K-means anomaly detection, and transfer learning techniques. Feature extraction will focus on six key network traffic attributes including packet duration, source bytes, destination bytes, protocol type, connection state, and service type. Model performance will be evaluated using confusion matrices and standard classification metrics, with data augmentation applied to improve accuracy.

The single-device architecture directly mirrors real-world cybersecurity deployments where low-cost embedded Linux devices monitor network traffic at the network perimeter, ensuring that all inference occurs entirely on-device without cloud dependency.

ML Application and Architecture

Problem Statement

Cyber threats continue to grow in both volume and sophistication, with network intrusion attacks representing one of the most significant risks to individuals and organisations alike. Traditional rule-based intrusion detection systems (IDS) rely on predefined signatures to identify malicious traffic; however, these systems struggle to detect novel or evolving attack patterns that fall outside known signatures (Moustafa & Slay, 2015). The challenge is compounded by the increasing volume of network traffic generated by connected devices, making cloud-based analysis both costly and latency-prone.

Machine learning offers a powerful alternative, enabling systems to learn patterns of normal behaviour from data and flag deviations without relying on static rules (Ray, 2022). Embedding this capability directly onto an edge device such as a Raspberry Pi 2 eliminates the need to transmit sensitive network data to the cloud for analysis, addressing both latency and privacy concerns simultaneously. As Doyu and Morabito (2019) describe, TinyML as-a-Service enables developers to introduce ML capabilities onto resource-constrained devices while delegating the computationally intensive training process to cloud infrastructure. The Raspberry Pi 2 platform was selected deliberately for its direct relevance to real-world cybersecurity deployments, where low-cost Linux-capable single-board computers are commonly used as network edge monitoring devices.

ML Task Definition

The ML application addresses a binary classification task. The model receives a feature vector derived from network traffic metadata as input and outputs one of two labels: Normal or Denial of Service (DoS). This is a supervised classification problem, where the model learns to distinguish between benign and malicious traffic patterns from labelled training data drawn from the UNSW-NB15 dataset (Moustafa & Slay, 2015). The Raspberry Pi 2 acts as the edge inference device, running the trained TinyML model and classifying incoming feature vectors in real time from the UNSW-NB15 test set. The trained model is exported as a Linux ARMv7 binary from Edge Impulse and executed entirely on the Raspberry Pi 2, with no internet connection required at inference time.

Figure 1

System Architecture for On-Device Network Traffic Anomaly Detection

[Figure 1 — Architecture Diagram to be inserted here: UNSW-NB15 Dataset → Edge Impulse Cloud Training → Linux ARMv7 Binary Export → Pi 2 (Edge Inference Device) → Classification Output: Normal / DoS]

Note. The figure illustrates the end-to-end system architecture of the proposed TinyML application. The UNSW-NB15 dataset is filtered and uploaded to Edge Impulse for cloud-based model training. The trained model is exported as a Linux ARMv7 binary and deployed to a Raspberry Pi 2 single-board computer acting as the edge inference device. At inference time, feature vectors derived from network traffic metadata are passed to the model, which returns

a real-time binary classification output of either Normal or Denial of Service (DoS). No internet connection is required at inference time. Adapted from Moustafa and Slay (2015) and Doyu and Morabito (2019).

Another learner reviewing this section should be able to understand that the application takes raw network traffic features as input on a Raspberry Pi 2, processes them through a trained Edge Impulse model running locally, and returns a real-time classification of whether the observed traffic is normal or a DoS attack.

Dataset and Acquisition Strategy

The UNSW-NB15 dataset, developed by the Australian Centre for Cyber Security at the University of New South Wales, will serve as the primary training and testing dataset for this project (Moustafa & Slay, 2015). The full dataset contains approximately 2.5 million records across nine attack categories; however, for this deployment a carefully selected subset will be used, consisting of approximately 800–1,000 records filtered to two classes — Normal and Denial of Service (DoS) — retaining six features: duration, protocol type, source bytes, destination bytes, connection state, and service type. This subset has been designed to operate within the constraints of Edge Impulse's free tier data limits while remaining representative of the core binary classification task.

Data will be downloaded in CSV format, filtered and pre-processed locally, and then uploaded into Edge Impulse with appropriate class labels applied. The opportunity to improve model accuracy through iterative data augmentation will be explored during the training phase, with additional synthetic samples generated for any underrepresented class to address potential class imbalance. The UNSW-NB15 dataset is publicly available at no cost through the University of New South Wales research portal, removing any data acquisition barrier entirely (Moustafa & Slay, 2015).

Table 1

Summary of the Filtered UNSW-NB15 Dataset Used for Model Training

Attribute	Detail
Dataset Name	UNSW-NB15
Source	Australian Centre for Cyber Security, University of New South Wales
Full Dataset Size	~2,540,044 records
Filtered Dataset Size	
Original Feature Count	49 features
Selected Feature Count	
Selected Features	

Traffic Classes (Full)	Normal, DoS, Probe, R2L, U2R, Backdoor, Analysis, Fuzzers, Reconnaissance, Shellcode
Traffic Classes (Filtered)	
Class Distribution (Normal)	
Class Distribution (DoS)	
Train Split	
Test Split	
File Format	CSV
Availability	Publicly available, free of charge
Access URL	https://research.unsw.edu.au/projects/unsw-nb15-dataset

Note. The table summarises the key attributes of the filtered subset of the UNSW-NB15 dataset selected for this project. The full UNSW-NB15 dataset contains approximately 2.5 million records across nine attack categories and 49 features; however, the subset used here has been filtered to two traffic classes — Normal and Denial of Service (DoS) — and reduced to six features to comply with Edge Impulse free tier data constraints while remaining representative of the core binary classification task. Dataset sourced from the Australian Centre for Cyber Security, University of New South Wales. From Moustafa and Slay (2015).

Model Training and Features

Feature Extraction

Feature extraction is a critical success factor in generating reliable ML models on resource-constrained edge devices such as the Raspberry Pi 2 (Ray, 2022). From the 49 available features in the UNSW-NB15 dataset, a reduced set of six features has been selected based on information gain and correlation analysis to minimise model size and inference latency on the ARMv7 architecture: duration, protocol type, source bytes, destination bytes, connection state, and service type. These tabular features will be uploaded to Edge Impulse and processed through a custom flatten processing block, which converts each feature vector into a format suitable for input into the neural network training block. Reducing the feature set to six inputs ensures the exported Linux binary remains lightweight enough to run efficiently on the Raspberry Pi 2's 1GB of RAM without performance degradation.

Learning Techniques

Neural Network Classification

A K-means anomaly detection block will be applied as a secondary layer of defence, targeting traffic patterns that deviate from established normal behaviour but may not match the DoS class specifically — effectively providing a mechanism for detecting novel or zero-day attack patterns. Unlike the supervised neural network, K-means operates in an unsupervised fashion, clustering normal traffic behaviour and flagging observations that deviate significantly from established cluster centroids as anomalous (Doyu & Morabito, 2019). This layered

approach — supervised classification combined with unsupervised anomaly detection — reflects the defence-in-depth principle central to professional cybersecurity practice.

K-Means Anomaly Detection

A K-means anomaly detection block will be applied as a secondary layer of defence, targeting zero-day threats that fall outside the labelled categories in the training data. Unlike the supervised neural network, K-means operates in an unsupervised fashion, clustering normal traffic behaviour and flagging observations that deviate significantly from established cluster centroids as anomalous (Doyu & Morabito, 2019). This is particularly valuable in a cybersecurity context where novel attack patterns may not yet be represented in any labelled dataset.

Transfer Learning

Transfer learning will be explored by fine-tuning a pre-trained Edge Impulse classification model on the filtered UNSW-NB15 feature set. This approach leverages representations already learned from related classification tasks, potentially improving convergence speed and accuracy particularly where the training subset is limited in size (Ray, 2022). Edge Impulse's EON Tuner tool will be used to identify the most efficient model architecture for the Raspberry Pi 2's ARMv7 processor, balancing classification accuracy against the memory and processing constraints of the hardware.

Figure 2

Edge Impulse Model Training and Deployment Pipeline

[Figure 2 — Training Pipeline Diagram to be inserted here: Filtered UNSW-NB15 CSV → Feature Selection (6 features) → Edge Impulse Flatten Block → Neural Network Training + K-Means Block → EON Tuner Optimisation → Linux ARMv7 Binary Export → Pi 2 Deployment]

Note. The figure illustrates the full model training and deployment pipeline used in this project. Filtered CSV data from the UNSW-NB15 dataset undergoes feature selection, retaining six attributes: duration, protocol type, source bytes, destination bytes, connection state, and service type. The selected features are processed through Edge Impulse's flatten processing block before being passed to the neural network training block and K-means anomaly detection block simultaneously. The EON Tuner optimisation tool is applied to identify the most efficient model architecture for the Raspberry Pi 2's ARMv7 processor. The finalised model is exported as a Linux ARMv7 binary and deployed directly to the Raspberry Pi 2 for on-device inference.

Evaluation of Model Performance and Improving Accuracy

Model performance will be evaluated using a held-out test split of the filtered UNSW-NB15 subset, representing 20% of the total records — approximately 160–200 samples. A confusion matrix will be generated for the initial benchmark model, providing a visual breakdown of true positives, false positives, true negatives, and false negatives across the Normal and DoS classes. Standard classification metrics including accuracy, precision, recall,

and F1-score will be calculated from the confusion matrix to provide a comprehensive picture of model performance (Moustafa & Slay, 2015).

Following the benchmark evaluation, data augmentation techniques will be applied to address any class imbalance between Normal and DoS records. A second confusion matrix will be generated post-augmentation and presented alongside the benchmark matrix to illustrate the improvement achieved. The Edge Impulse Analyse Optimisation tool will be used to assess the trade-off between model accuracy and on-device resource consumption, with particular attention paid to RAM usage and inference latency on the Raspberry Pi 2's ARMv7 architecture, ensuring the deployed model meets the real-time classification requirements of the two-device system.

Figure 3

Confusion Matrices for Benchmark Model and Post-Augmentation Model

[Figure 3 — Side-by-side Confusion Matrices to be inserted here: Benchmark | Post-Augmentation]

Note. The figure presents two confusion matrices displayed side by side for comparative evaluation of model performance. The left matrix shows the benchmark classification results for the initial trained neural network model against the held-out UNSW-NB15 test set, prior to any data augmentation. The right matrix shows the classification results following data augmentation applied to address class imbalance between Normal and DoS traffic records. Each matrix displays the number of true positives, true negatives, false positives, and false negatives across the two target classes: Normal and Denial of Service (DoS). Improvements in true positive rate and reduction in false negatives between the two matrices indicate the effect of augmentation on model performance. Values to be populated following Edge Impulse model training.

Table 2

Model Performance Metrics for Benchmark and Post-Augmentation Neural Network Classifier

Attribute	Description	Benchmark Model	Post-Augmentation Model
Accuracy	Overall proportion of correctly classified records	%	%
Precision	Proportion of DoS predictions that were correct	%	%
Recall	Proportion of actual DoS records correctly identified	%	%
F1-Score	Harmonic mean of precision and recall	%	%
Inference Time (ms)	Average time to classify one feature vector on Pi 2	%	%

RAM Usage (KB)	Peak memory consumption during inference on Pi 2	%	%
----------------	---	---	---

Note. The table presents six performance metrics evaluated against the held-out UNSW-NB15 test set for both the initial benchmark model and the post-augmentation model. Accuracy reflects the overall proportion of correctly classified records. Precision indicates the proportion of DoS predictions that were correct. Recall reflects the proportion of actual DoS records correctly identified by the model — a critical metric in a cybersecurity context where false negatives carry the highest consequence. F1-score provides the harmonic mean of precision and recall. Inference time reflects the average time in milliseconds for the model to classify a single feature vector on the Raspberry Pi 2's ARMv7 processor. RAM usage reflects the peak memory consumption of the model during inference. All values to be populated following Edge Impulse model training and Raspberry Pi 2 deployment.

Risks, Limitations and Advantages

Risks and Limitations

Several risks and limitations are inherent in deploying a TinyML-based intrusion detection model on a Raspberry Pi 2. First, as with all ML systems, the model outputs probabilistic estimates rather than definitive classifications (Bier, 2020). A false negative — where a genuine DoS attack is classified as normal traffic — represents the most serious failure mode in a cybersecurity context, as it would leave the monitored network unprotected. Second, the model is trained on a static subset of the UNSW-NB15 dataset and may exhibit performance degradation over time as new attack patterns emerge that were not present in the training data, a phenomenon known as concept drift (Ray, 2022). Third, the deliberate reduction to six features and approximately 800–1,000 training records, while necessary for Edge Impulse's free tier constraints, limits the breadth of attack patterns the model can reliably detect compared to a full-scale deployment.

The Raspberry Pi 2's ARMv7 architecture and 1GB of RAM impose real hardware constraints on model complexity, and the absence of built-in WiFi on the Pi 2 requires a USB WiFi dongle or ethernet connection during the initial setup and model deployment phase (Bier, 2020). Additionally, the two-device simulation architecture, while demonstrative of real-world deployment principles, does not replicate the full complexity of live network traffic monitoring.

Advantages

The advantages of on-device ML inference are particularly compelling in a cybersecurity application and are well-captured by the BLERP framework (Bier, 2020).

Processing occurs entirely on the Raspberry Pi 2, eliminating the bandwidth overhead of streaming network traffic to the cloud. Inference operates in near real time on the ARMv7 hardware, satisfying the low-latency requirement of intrusion detection. Repurposing existing Raspberry Pi 2 hardware incurs zero additional cost, addressing the economics dimension. The model continues to function without internet connectivity once deployed, ensuring reliability even during network outages — a scenario that may itself be the result of an active attack. Most critically, sensitive network traffic data never leaves the device, directly addressing the privacy dimension in a context where data confidentiality is paramount (Doyu & Morabito, 2019). The Raspberry Pi 2 deployment further demonstrates that this solution is scalable — additional Pi units could be deployed at multiple network perimeter points simultaneously, each running independent inference at minimal cost.

Project Plan

Table 3
Machine Learning Project Plan with Task Descriptions and Target Completion Weeks

Task ID	Task Description	Target Week
0	Set up Raspberry Pi 2 hardware, flash OS, install Edge Impulse Linux runner, verify device connection	5
1		5
2	Download UNSW-NB15, filter to Normal/DoS, select 6 features, trim to 800–1,000 rows	5
3		5
4		6
5	Review feature space in Edge Impulse feature explorer; verify class separability via UMAP	7
6		7
7	Explore transfer learning via EON Tuner; augment data; retrain to improve accuracy	8
8	Export Linux ARMv7 binary; deploy to Pi 2; run initial inference test against UNSW-NB15 test set	9
9	Run live inference test on Pi 2; record video presentation	10
10	Final write-up, code commenting and submission packaging	11

Note. The table outlines the ten key tasks required to complete the machine learning prototype from initial setup through to final submission. Tasks are sequenced to reflect the

natural dependencies of the Edge Impulse TinyML development workflow: data acquisition and preprocessing precede model training, which precedes optimisation, deployment, and evaluation. Target weeks are aligned to ensure all tasks are completed by Week 10, providing a one-week buffer before the final submission deadline. Task durations are estimates and may be adjusted based on model training outcomes and hardware deployment results.

Investment

This project repurposes an existing Raspberry Pi 2 single-board computer acquired previously, incurring zero additional hardware cost. The Raspberry Pi 2 was selected deliberately as the deployment target for its direct relevance to real-world cybersecurity deployments, where low-cost embedded Linux devices are commonly used to monitor network traffic at the perimeter. A Raspberry Pi 2 retails for approximately AUD \$45–60; however, as the unit is already owned, no financial investment is required for hardware. A USB WiFi dongle or ethernet connection will be required for initial setup and model deployment, which is also available at no additional cost using existing equipment. The Edge Impulse developer account is free of charge, and the UNSW-NB15 dataset is publicly available at no cost, making the total project investment zero. It should be noted that upgrading to a Raspberry Pi 4 (approximately AUD \$100–130) would offer improved inference speed and built-in WiFi, but is not necessary for the scope of this prototype.

Responsible Artificial Intelligence

Responsible AI licensing will be applied to this project to ensure the model cannot be repurposed for harmful or unlawful applications. A Responsible AI License v0.1, available at <https://www.licenses.ai/ai-licenses/>, will be applied to both the end-user deployment and the source code. Specific restrictions will prohibit the use of this model for mass surveillance, unlawful interception of private communications, or the profiling of individuals without their consent — all of which are particularly salient concerns given the network traffic data this model processes. Beyond restriction, this application is actively aligned with social good: empowering individuals and organisations to detect cyber threats locally, privately, and without reliance on third-party cloud services that may themselves represent a security risk (Doyu & Morabito, 2019). The open-source nature of both the Raspberry Pi platform and the Edge Impulse WebAssembly export further ensures that this solution remains transparent, auditable, and accessible to the wider security research community.

References

- Benzion, A. (2020, September 3). TinyML is breathing life into billions of devices. *TNW*.
<https://thenextweb.com/neural/2020/09/03/tinyml-is-breathing-life-into-billions-of-devices/>
- Bier, J. (2020, September 6). AI and vision at the edge. *EE Times*.
<https://www.eetimes.com/ai-and-vision-at-the-edge/#>
- Doyu, H., & Morabito, R. (2019, December 13). TinyML as-a-service: What is it and what does it mean for the IoT edge? *Ericsson Blog*.
<https://www.ericsson.com/en/blog/2019/12/tinyml-as-a-service-iotedge>
- Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). *2015 Military Communications and Information Systems Conference (MilCIS)*, 1–6.
<https://doi.org/10.1109/MilCIS.2015.7348942>
- Ray, P. P. (2022). A review on TinyML: State-of-the-art and prospects. *Journal of King Saud University — Computer and Information Sciences*, 34(4), 1595–1623.
<https://doi.org/10.1016/j.jksuci.2021.11.019>